



Chiang Mai University

AnyaspyAC

Suthana Kwaonueng, Theerada Siri, Sorawach Utas

2026-07-13

1 Contest

2 Tanya

3 cellul4r

4 Ohm

Contest (1)

template.cpp80 lines

```
#include<bits/stdc++.h>
using namespace std;
void __print(int x) {cerr << x;}
void __print(long x) {cerr << x;}
void __print(long long x) {cerr << x;}
void __print(unsigned x) {cerr << x;}
void __print(unsigned long x) {cerr << x;}
void __print(unsigned long long x) {cerr << x;}
void __print(float x) {cerr << x;}
void __print(double x) {cerr << x;}
void __print(long double x) {cerr << x;}
void __print(char x) {cerr << '\'' << x << '\'';}
void __print(const char *x) {cerr << '\"' << x << '\"'}
void __print(const string &x) {cerr << '\"' << x << '\"'}
void __print(bool x) {cerr << (x ? "true" : "false");}

template<typename T, typename V>
void __print(const pair<T, V> &x);
template<typename T>
void __print(const T &x) {int f = 0; cerr << '{'; for (auto &i:
x) cerr << (f++ ? ", " : ""), __print(i); cerr << "}";}
template<typename T, typename V>
void __print(const pair<T, V> &x) {cerr << '{'; __print(x.first
); cerr << ", "; __print(x.second); cerr << '}';}
void __print() {cerr << "]\n";}
template <typename T, typename... V>
void __print(T t, V... v) {__print(t); if (sizeof...(v)) cerr <<
", "; __print(v...);}

#ifdef DEBUG
#define dbg(x...) cerr << "\e[91m"<<__func__<<": "<<__LINE__<<"
[" << #x << "] = ["; __print(x); cerr << "\e[39m" << endl;
#else
#define dbg(x...)
#endif

typedef long long ll;
typedef long double ld;
typedef complex<ld> cd;

typedef pair<int, int> pi;
typedef pair<ll,ll> pl;
typedef pair<ld,ld> pd;

typedef vector<int> vi;
typedef vector<ld> vd;
typedef vector<ll> vl;
typedef vector<pi> vpi;
typedef vector<pl> vpl;
typedef vector<cd> vcd;
```

```
//template<class T> using pq = priority_queue<T>;
//template<class T> using pqg = priority_queue<T, vector<T>,
greater<T>>;
```

```
#define rep(i, a) for(int i=0;i<a;++i)
#define FOR(i, a, b) for (int i=a; i<(b); i++)
#define F0R(i, a) for (int i=0; i<(a); i++)
#define FORd(i,a,b) for (int i = (b)-1; i >= a; i--)
#define F0Rd(i,a) for (int i = (a)-1; i >= 0; i--)
#define trav(a,x) for (auto& a : x)
#define uid(a, b) uniform_int_distribution<long long>(a, b)(rng
)

#define sz(x) (int)(x).size()
#define mp make_pair
#define pb push_back
#define f first
#define s second
#define lb lower_bound
#define ub upper_bound
#define all(x) x.begin(), x.end()
#define ins insert
```

```
template<class T> bool ckmin(T& a, const T& b) { return b < a ?
a = b, 1 : 0; }
template<class T> bool ckmax(T& a, const T& b) { return a < b ?
a = b, 1 : 0; }
```

```
mt19937 rng(chrono::steady_clock::now().time_since_epoch().
count());
```

```
const char nl = '\n';
const int N = 2e5+3;
const int INF = INT_MAX;
const long long LINF = LLONG_MAX;
```

```
int main(){
ios::sync_with_stdio(false);cin.tie(nullptr);
}
```

.vimrc21 lines

```
"General editor settings
set tabstop=4
set nocompatible
set shiftwidth=4
set expandtab
set autoindent
set smartindent
set ruler
set showcmd
set incsearch
set shellslash
set number
set relativenumber
set cino+=L0
```

```
"keybindings for { completion, "jk" for escape, ctrl-a to
select all
inoremap {<CR> {<CR><Esc>O
inoremap {} {}
imap jk <Esc>
map <C-a> <esc>ggVG<CR>
set belloff=all
```

.bashrc1 lines

```
export PATH=$PATH:~/scripts/
```

build.sh1 lines

```
g++ -static -DLOCAL -lm -s -x c++ -Wall -Wextra -O2 -std=c++17
-o $1 $1.cpp
```

stress.sh22 lines

```
#!/usr/bin/env bash

for ((testNum=0;testNum<$4;testNum++))
do
./$3 > input
./$2 < input > outSlow
./$1 < input > outWrong
H1=`md5sum outWrong`
H2=`md5sum outSlow`
if !(cmp -s "outWrong" "outSlow")
then
echo "Error found!"
echo "Input:"
cat input
echo "Wrong Output:"
cat outWrong
echo "Slow Output:"
cat outSlow
exit
fi
done
echo Passed $4 tests
```

Tanya (2)

2.1 Template & Utilities

pragma.hDescription: random pragma magic that I don't fucking know but ocasionally solve my stupid bad implementation code1021ce, 7 lines

```
//some one on cf told us that these are mostly enough
#pragma GCC target ("avx2")
#pragma GCC optimize ("O3")
#pragma GCC optimize ("unroll-loops")
//just in case have this one
#pragma GCC optimize("O3,unroll-loops")
#pragma GCC target ("avx2,bmi,bmi2,popcnt,lzcnt")
```

cellul4r (3)

Ohm (4)

Techniques (A)

techniques.txt	159 lines
Recursion	
Divide and conquer	
Finding interesting points in N log N	
Algorithm analysis	
Master theorem	
Amortized time complexity	
Greedy algorithm	
Scheduling	
Max contiguous subvector sum	
Invariants	
Huffman encoding	
Graph theory	
Dynamic graphs (extra book-keeping)	
Breadth first search	
Depth first search	
* Normal trees / DFS trees	
Dijkstra's algorithm	
MST: Prim's algorithm	
Bellman-Ford	
Konig's theorem and vertex cover	
Min-cost max flow	
Lovasz toggle	
Matrix tree theorem	
Maximal matching, general graphs	
Hopcroft-Karp	
Hall's marriage theorem	
Graphical sequences	
Floyd-Warshall	
Euler cycles	
Flow networks	
* Augmenting paths	
* Edmonds-Karp	
Bipartite matching	
Min. path cover	
Topological sorting	
Strongly connected components	
2-SAT	
Cut vertices, cut-edges and biconnected components	
Edge coloring	
* Trees	
Vertex coloring	
* Bipartite graphs (=> trees)	
* 3^n (special case of set cover)	
Diameter and centroid	
K'th shortest path	
Shortest cycle	
Dynamic programming	
Knapsack	
Coin change	
Longest common subsequence	
Longest increasing subsequence	
Number of paths in a dag	
Shortest path in a dag	
Dynprog over intervals	
Dynprog over subsets	
Dynprog over probabilities	
Dynprog over trees	
3^n set cover	
Divide and conquer	
Knuth optimization	
Convex hull optimizations	
RMQ (sparse table a.k.a 2^k-jumps)	
Bitonic cycle	
Log partitioning (loop over most restricted)	
Combinatorics	

Computation of binomial coefficients
Pigeon-hole principle
Inclusion/exclusion
Catalan number
Pick's theorem
Number theory
Integer parts
Divisibility
Euclidean algorithm
Modular arithmetic
* Modular multiplication
* Modular inverses
* Modular exponentiation by squaring
Chinese remainder theorem
Fermat's little theorem
Euler's theorem
Phi function
Frobenius number
Quadratic reciprocity
Pollard-Rho
Miller-Rabin
Hensel lifting
Vieta root jumping
Game theory
Combinatorial games
Game trees
Mini-max
Nim
Games on graphs
Games on graphs with loops
Grundy numbers
Bipartite games without repetition
General games without repetition
Alpha-beta pruning
Probability theory
Optimization
Binary search
Ternary search
Unimodality and convex functions
Binary search on derivative
Numerical methods
Numeric integration
Newton's method
Root-finding with binary/ternary search
Golden section search
Matrices
Gaussian elimination
Exponentiation by squaring
Sorting
Radix sort
Geometry
Coordinates and vectors
* Cross product
* Scalar product
Convex hull
Polygon cut
Closest pair
Coordinate-compression
Quadtrees
KD-trees
All segment-segment intersection
Sweeping
Discretization (convert to events and sweep)
Angle sweeping
Line sweeping
Discrete second derivatives
Strings
Longest common substring
Palindrome subsequences

Knuth-Morris-Pratt
Tries
Rolling polynomial hashes
Suffix array
Suffix tree
Aho-Corasick
Manacher's algorithm
Letter position lists
Combinatorial search
Meet in the middle
Brute-force with pruning
Best-first (A*)
Bidirectional search
Iterative deepening DFS / A*
Data structures
LCA (2^k-jumps in trees in general)
Pull/push-technique on trees
Heavy-light decomposition
Centroid decomposition
Lazy propagation
Self-balancing trees
Convex hull trick (wcipeg.com/wiki/Convex_hull_trick)
Monotone queues / monotone stacks / sliding queues
Sliding queue using 2 stacks
Persistent segment tree